



Module 3 — Data Modeling

- Master Relational Data Modeling, ERD Design, Normalization, Star & Snowflake Schemas, Data Integrity, Performance Optimization, and Real-World Case Studies.

**Author
Srinivas Dande**





Module 3 — Data Modeling

1. Foundations of Data Modeling

- 1.1. Introduction to Data Models
- 1.2. OLTP vs OLAP
- 1.3. ACID Properties
- 1.4. BASE Properties (NoSQL)
- 1.5. ACID Properties Vs BASE Properties

2. Entity-Relationship Diagrams (ERDs)

- 2.1. Introduction to ERDs
- 2.2. Core Components of ERDs
- 2.3. Converting Requirements → ERD
- 2.4. Real-world Example: Mini Food Delivery System
- 2.5. Step-by-step: How to Draw in Draw.io

3. Normalization & Normal Forms

- 3.1. What is Normalization?
- 3.2. Functional Dependencies (FDs)
- 3.3. First Normal Form (1NF)
- 3.4. Second Normal Form (2NF)
- 3.5. Third Normal Form (3NF)
- 3.6. BCNF (Boyce–Codd Normal Form)
- 3.7. SQL Denormalization
- 3.8. ASSIGNMENT – Food Delivery Normalization



4. Data Warehouse Design

- 4.1. Why Data Warehouses?
- 4.2. Facts & Dimensions
- 4.3. Star Schema
- 4.4. Snowflake Schema
- 4.5. Star Schema vs Snowflake Schema
- 4.6. Mini Case Study — Star Schema
- 4.7. Mini Case Study — Snowflake Schema

5) Database Optimization

- 5.1. Database Partitioning
- 5.2. Data Sharding
- 5.3. Performance Anti-Patterns



1. Foundations of Data Modeling

1.1. Introduction to Data Models

1.2. OLTP vs OLAP

1.3. ACID Properties

1.4. BASE Properties (NoSQL)

1.5. ACID Properties Vs BASE Properties



1.1. Introduction to Data Models

A) What is a Data Model?

- Data Model is a structured way of representing Data so that both humans and machines can understand how information is stored, connected, and processed.
- **Data model defines:**
 - How data is stored
 - How different data elements are related
 - How data can be accessed and used
 - Rules that control the data
- In simple words:
 - Data Model is a blueprint for organizing data in a database.

B) Why Do We Need Data Modeling?

- Real-world systems (like food delivery, banking etc) handle large volumes of data.
- Without proper modeling, systems become Slow, Inconsistent and Difficult to maintain
- **Without modeling:**
 - Data becomes unstructured
 - Relationships are unclear
 - Queries become slow
 - Duplicate data creates inconsistencies
 - System becomes difficult to scale
- **With proper modeling:**
 - Data is clean, structured, and consistent
 - Systems perform faster
 - Developers understand the system easily
 - Analytics, reporting, and ML become easier
 - System is easier to maintain and scale
- **Data modeling ensures quality, performance, and scalability.**



C) Types of Data Models

- There are 3 main levels of data modeling:
 - 1) Conceptual Data Model
 - 2) Logical Data Model
 - 3) Physical Data Model

1) Conceptual Data Model

- High-level business view
- Identifies entities and relationships
- No technical details

Example: Customer, Orders, Restaurant

2) Logical Data Model

- More detailed
- Defines attributes, primary keys, foreign keys, cardinality
- Still not tied to a specific database

3) Physical Data Model

- Database implementation-level
- Defines data types, indexes, constraints, table storage, partitioning

✓ This hierarchy helps move from **business** → **technical** → **implementation**.



D) Relational Data Models (SQL Based)

- Relational data model is the most popular and widely used model in the world.
- It organizes data into:
 - Tables
 - Rows
 - Columns
 - Relationships
- **Examples of RDBMS:**
 - MySQL
 - PostgreSQL
 - Oracle
 - MS SQL Server
 - SQLite etc
- **Why relational model is dominant:**
 - Structured and consistent
 - Uses SQL (universal query language)
 - Supports ACID transactions
 - Easy to model business processes
 - Best suited for OLTP systems (banking, e-commerce, apps)

E) Non-Relational Data Models (NoSQL Based)

- NoSQL databases are designed to store large, flexible, and unstructured data.
- They do not require fixed schemas and scale horizontally with ease.

- Types of NoSQL Data Models:

- **a) Document Model**

- Stores JSON-like documents

Examples:

- MongoDB
- CouchDB

- **b) Key-Value Model**

- Data stored as key → value pairs

Examples:

- Redis
- DynamoDB

- **c) Column-Family Model**

- Column-oriented storage, suitable for large-scale analytics

Examples:

- Cassandra
- HBase

- **d) Graph Model**

- Stores nodes and edges to represent relationships

Examples:

- Neo4j
- Amazon Neptune

■ **Why NoSQL is popular:**

- Flexible schema
- Handles large-scale, high-velocity data
- Easy horizontal scaling
- BASE transactions (high availability, partition-tolerant)
- Handles unstructured and semi-structured data
- High write and read performance
- Designed for large-scale, high-velocity data
- Best for OLAP workloads in real-time analytics

■ **Ideal for:**

- Social Networks
- Real-time Analytics
- Logging Systems
- Recommendation Engines,
- IoT Data
- High-Traffic Applications.

■ **In summary:**

- NoSQL focuses on **flexibility**, **scalability**, and **high performance**.



1.2. OLTP vs OLAP

- Modern applications deal with two very different kinds of data processing:
 - OLTP (Online Transaction Processing)
 - OLAP (Online Analytical Processing)
- Understanding the difference is critical for
 - Designing Scalable Databases
 - Proper Data Models.

A) What is OLTP?

- OLTP = Real-time transactional systems
- Systems where users perform frequent small operations such as inserts, updates, and deletes.
- **Characteristics of OLTP:**
 - Supports high number of concurrent users
 - Stores current, real-time data
 - Highly normalized (1NF → BCNF)
 - Fast reads AND writes
 - Handles transactions (ACID)
 - Very strict on data integrity
- **Typical OLTP Operations:**
 - Place an order
 - Update profile
 - Transfer money
 - Book a ticket
- **Examples:**
 - Banking apps
 - E-commerce checkout
 - Swiggy/Zomato order placement
 - Railway/Flight booking
 - Hospital registration system
- **Purpose:**
 - Make transactions fast, accurate, and reliable.



B) What is OLAP?

- OLAP = Analytical systems for reporting & decision-making
- Systems where users run large, complex queries to analyze data.
- **Characteristics of OLAP:**
 - Stores huge historical datasets
 - Used for analytics and business intelligence (BI)
 - Data is denormalized (Star / Snowflake schema)
 - Optimized for read-heavy workloads
 - Few users, but heavy queries
 - Summaries, aggregates, trends
- **Typical OLAP Operations:**
 - Show monthly sales
 - Top 10 restaurants in Bangalore
 - Average delivery time
 - Customer churn trends
 - Year-over-year revenue growth
- **Examples:**
 - Data warehouses
 - Power BI dashboards
 - Analytics tools
 - Marketing insights
 - Sales forecasting systems
- **Purpose:**
 - Make reporting & analytics fast and efficient.



C) Quick Real-Life Examples

- **OLTP Example (Swiggy Order Placement):**
 - Srinivas orders a Biryani
 - **Database inserts:**
 - ✓ Customer record
 - ✓ Order record
 - ✓ Order items
 - ✓ Payment status

- **OLAP Example (Swiggy Business Dashboard):**
 - Management wants:
 - ✓ What are top 10 selling items this month?
 - ✓ Compare North vs South India order volume

This is OLAP analysis on historical fact tables.

D) Key Differences: OLTP vs OLAP

Feature	OLTP	OLAP
Purpose	Daily operations	Analytics & reporting
Data	Current, real-time	Historical, aggregated
Model	Normalized (1NF–BCNF)	Denormalized (Star/Snowflake)
Workload	Short transactions	Complex queries
Users	Thousands	Few analysts
Writes	Heavy	Light
Reads	Light to moderate	Heavy, large scans
Integrity	Very strict (ACID)	Flexible(BASE)
Examples	Swiggy orders	Swiggy monthly sales dashboard



1.3. ACID Properties

- **What is a Transaction?**
 - ✓ Transaction is a group of operations executed as a single logical unit.
 - ✓ **Example (Swiggy order payment):**
 - Deduct amount from wallet / card
 - Create the order
 - Generate invoice
 - Update restaurant queue
 - If any step fails → the entire transaction must fail.
 - This is where ACID comes in.
- ACID stands for
 - Atomicity
 - Consistency
 - Isolation
 - Durability.

A) Atomicity — “All or Nothing”

- Atomicity ensures that either
 - ✓ **Entire transaction succeeds or none of it happens.**
 - ✓ If all steps success → commit
 - ✓ If any step fails → rollback
- **Example: Money Transfer**
 - ✓ Customer A transfers ₹5000 to Customer B:
 - Deduct ₹5000 from A
 - Add ₹5000 to B
 - ✓ **If Step 2 fails → Step 1 is reversed → no money is lost.**
- **Without atomicity,**
 - ✓ Users lose money or get double debited.



B) Consistency — “Valid State → Valid State”

- Consistency ensures that
 - ✓ **Database moves from one valid state to another valid state.**
- Rules, Constraints must always hold true.
- **Example: Money Transfer**
 - ✓ If total balance across A & B = ₹10,000 before transaction
 - ✓ It must remain ₹10,000 after transaction.
 - ✓ No money should disappear or be created magically.
 - Users lose money or get double debited.
- **Consistency = No corruption, No invalid data.**

C) Isolation — “Transactions Don’t Disturb Each Other”

- Isolation ensures that
 - ✓ **Concurrent Transactions do not affect each other.**
- **Example:**
 - ✓ Two customers withdraw money at the same time from the same account.
 - ✓ Isolation ensures:
 - One withdrawal is processed at a time
 - Final balance is always correct
 - No dirty reads
 - No lost updates
- **Without isolation**
 - ✓ Simultaneous operations create incorrect results.



D) Durability — “Committed Means Permanent”

- Once a transaction is committed,
 - ✓ **Changes are permanently saved even if:**
 - System crashes
 - Power goes off
 - Server restarts
- **Example:**
 - ✓ After money is transferred successfully:
 - Updated balances are stored safely
 - Even after restart → data is preserved
 - No rollback after commit
- **Durability gives trust:**
 - ✓ If it succeeded, it will stay

ACID Summary

Property	Meaning	Summary
Atomicity	All steps succeed or none	No partial updates
Consistency	Valid state → Valid state	No corruption or No Invalid State
Isolation	No interference	Safe concurrency
Durability	Permanent commit	Crash-proof



1.4. BASE Properties (NoSQL)

- BASE is the transaction model followed by many NoSQL databases.
- It is the opposite of ACID and is designed for
 - ✓ High Availability
 - ✓ Horizontal Scaling
 - ✓ Distributed Systems.
- BASE stands for
 - Basically Available
 - Soft State
 - Eventually Consistent
- Unlike ACID (strict, guaranteed correctness),
 - ✓ BASE relaxes consistency to gain speed and scalability.

A) Basically Available — “System is Always On”

- System guarantees availability even during:
 - ✓ **Node failures**
 - ✓ **Network partitions**
 - ✓ **High traffic**
- Instead of rejecting requests, the system continues to operate.
- **Example:**
 - ✓ In a distributed shopping cart system:
 - Even if one node fails
 - Users can still add items to cart (handled by another node)
- **In Short:**
 - ✓ System always responds.



B) Soft State — “State Can Change Over Time”

- Data stored in a Node.
 - ✓ **may change even without any new input**
- This happens because:
 - ✓ Replication is asynchronous
 - ✓ Different nodes may update at different times
- Soft state accepts temporary inconsistency inside the cluster.
- **Example:**
 - ✓ In a multi-region NoSQL database:
 - US region updates now
 - India region receives update a few milliseconds later
 - Data is temporarily different across nodes.
- **In Short:**
 - ✓ Temporary inconsistency is allowed.

C) Eventually Consistent — “Consistency After Some Time”

- System does not guarantee immediate consistency.
 - ✓ **But all replicas will become consistent eventually**
- **Example:**
 - ✓ If a user updates their profile:
 - One replica updates instantly
 - Other replicas sync the update later
 - After some time → all nodes have the same value
- **This enables:**
 - ✓ High speed
 - ✓ High availability
 - ✓ Fault tolerance
- **In Short:**
 - ✓ Final consistency is guaranteed, not immediate consistency.



1.5. ACID vs BASE

Feature	ACID	BASE
Used In	SQL Databases	NoSQL Databases
Consistency	Strong, Immediate	Eventual Consistency
Availability	Moderate	Very High
Scalability	Vertical Scaling (Scale-Up)	Horizontal Scaling (Scale-Out)
Integrity	Very Strict	Flexible
Typical Systems	OLTP, Banking, E-commerce	Big Data, Distributed Apps, Analytics



2. Entity-Relationship Diagrams (ERDs)

2.1. Introduction to ERDs

2.2. Core Components of ERDs

2.3. Converting Requirements → ERD

2.4. Real-world Example: Mini Food Delivery System

2.5. Step-by-step: How to Draw in Draw.io



2.1. Introduction to ERDs

A) What is an ERD (Entity-Relationship Diagram)?

- An ERD is a visual diagram that shows:
 - Entities (tables or real-world objects)
 - Attributes (fields/columns describing each entity)
 - Relationships between entities
- ERDs help you understand, design, and communicate the **logical structure** of a database before **creating physical tables**.
- It is the foundation of database design and converts business requirements into a clear, logical data model.

B) Why ERDs are used in real projects

ERDs are used because of the following reasons

- **Convert business requirements → data structure**
 - Identify entities, attributes, relationships.
- **Avoid database design mistakes early**
 - Wrong cardinality, missing relationships, poor normalization.
- **Improve communication**
 - Used in discussions between developers, architects, business analysts, and Database Administrators (DBAs).
- **Help in normalization & schema design**
 - ERD acts as the blueprint for physical tables.
- **Support future scalability**
 - Good ERD = fewer joins, fewer schema changes, easier sharding later.



C) ERD vs Schema Diagram

- **1. ERD (Entity-Relationship Diagram)**
 - **Focus:** Data & relationships
 - **Shows:** Entities, attributes, PK/FK, cardinality
 - **Used in:** Databases, backend systems
- **2. Schema Diagram**
 - **Focus:** Actual SQL tables
 - **Shows:** Tables, columns, data types, constraints
 - **Represents** the physical database structure based on ERD

ERD = Logical

Schema Diagram = Physical

D) Tools for Drawing ERDs

- **Simple Online Tools**
 - Draw.io
 - Lucidchart
 - ERDPlus (free academic tool)
 - DBDiagram.io
 - SQLDBM
- **Database Tools**
 - MySQL Workbench
 - PostgreSQL PgAdmin ERD Generator



2.2. Core Components of ERDs

- List of Core Components of ERDs
 - 1) Entities
 - 2) Attributes
 - 3) Keys
 - 4) Relationships
 - 5) Cardinality
 - 6) Participation (Optional / Mandatory)
 - 7) Identifying vs Non-identifying Relationships
 - 8) Crow's Foot Notation

1. Entities

- Real-world objects you want to store data for.
- Examples:
 - Customer, Order, Product, Driver, Restaurant

2. Attributes

- Properties that describe an entity.
- Examples:
 - name, email, phone, price, city
- Attribute Types
 - Simple → indivisible (price, age)
 - Composite → can be split (address → street, city, pincode)
 - Single-valued → one value
 - Multi-valued → many values (skills, multiple phone numbers) → avoid in SQL
 - Derived → computed (age from DOB)

3. Keys

- Used for identification and linking.
- Primary Key (PK) → Unique record identifier
- Foreign Key (FK) → Creates relationship between two tables
- Composite Key → Combination of columns (e.g., order_id + food_id in OrderItem)



4. Relationships

- **Connections between entities.**
- 3 Types of Relationships
- **One-to-One (1:1)**
 - User ↔ UserProfile
- **One-to-Many (1:N)**
 - Customer → Orders
 - Restaurant → MenuItems
- **Many-to-Many (M:N)**
 - Students ↔ Courses
 - Products ↔ Categories
 - (Resolved using a bridge table)
- **Self-Referencing**
 - Employee → Manager
 - Category → Parent Category
 - Entity relates to itself
 - MySQL Workbench
 - PostgreSQL PgAdmin ERD Generator

5. Cardinality

- **Defines how many of each side can relate.**
 - Minimum Cardinality: 0 or 1
 - Maximum Cardinality: 1 or many
- **Examples:**
 - One user can have 0 or many orders → (0, N)
 - One course can have 0 or many student enrollments → (0, N)
 - One order must have at least 1 item → (1, N)
 - One department must have at least 1 employee → (1, N)



6. Participation (Optional / Mandatory)

- **Mandatory Participation:**
 - ✓ Relationship must exist
 - **Example:**
 - Order → must belong to a Customer
 - Employee → must belong to a Department
 - Invoice → must be issued for a Purchase
- **Optional Participation:**
 - ✓ Relationship may or may not exist
 - **Example:**
 - Customer may have 0 orders
 - Driver may have 0 Assigned Vehicles
 - Student may have 0 Scholarships

7. Identifying vs Non-identifying Relationships

- **Identifying Relationship (Strong):**
 - ✓ Child cannot exist without parent
 - ✓ FK is mandatory
 - ✓ **Example:**
 - OrderItem → Order
 - **Why is it identifying?**
 - OrderItem cannot exist without an Order.
 - Every order item must be linked to exactly one order.
- **Non-identifying Relationship (Weak):**
 - ✓ Child can exist independently
 - ✓ FK not part of PK
 - ✓ **Example:**
 - Product → Category
 - **Why Non-identifying?**
 - Product can exist without a Category (optional).
 - Product has its own identity → product_id (PK).
 - category_id is a normal FK (not part of PK).



8. Crow's Foot Notation

- **Symbols:**

| : one

0 : zero

< : many

- **Examples:**

1:N → |——<

0:N → 0——<

1:1 → |——|

- **Most commonly used in ER diagrams.**



2.3. Converting Requirements → ERD

Step 1: Extracting Entities

- From the problem statement, **identify nouns** → usually become entities.
- Example (Swiggy):
 - Customer
 - Restaurant
 - Order
 - OrderItem
 - FoodItem
- Rule:
 - If something needs to store data → it's an entity.

Step 2: Identifying Relationships

- Look for **verbs or actions** between entities → these become relationships
- Examples:
 - Customer places Order
 - Restaurant prepares Order
 - Order contains OrderItems
 - OrderItem refers to FoodItem
- Verb = relationship
- **Direction + cardinality = business rules**



Step 3: Picking Correct Cardinality

- Based on business logic:
- **1:1:**
 - One Delivery Partner ↔ One Vehicle
- **1:N (most common):**
 - One Customer → Many Orders
 - One Restaurant → Many FoodItems
- **M:N:**
 - FoodItems ↔ Add-ons
 - Implement using a Bridge Table

Step 4: Identifying Primary Keys

- Every entity must have a unique identifier.
- **Natural Keys:**
 - Natural key is a real-world, meaningful attribute that uniquely identifies a record in the business domain.
 - Examples:
 - email
 - phone
 - product_code
- **Surrogate Keys:**
 - ✓ Surrogate key is an artificial, system-generated identifier used only inside the database (not meaningful in real life).
 - ✓ **Examples:**
 - customer_id
 - order_id
 - item_id

Best Practice:

- ✓ Use Surrogate Keys (Auto-increment / UUID) for consistency & scalability.

Step 5: Normalizing Attributes

- Each attribute must hold one atomic value → 1NF
- No repeating groups, no multi-valued attributes



2.4. Real-world Example: Mini Food Delivery System

Step 1 – List Entities

- List of Entities in Food Delivery System
 - CUSTOMER
 - RESTAURANT
 - FOOD_ITEM
 - ORDER
 - ORDER_ITEM

Step 2 – Sample Attributes

- Entities with Sample Attributes
- **Entity : Customer**
 - customer_id (PK)
 - name
 - phone
 - email
 - city
- **Entity : Restaurant**
 - restaurant_id (PK)
 - name
 - location
 - rating
- **Entity : FoodItem**
 - food_id (PK)
 - restaurant_id (FK)
 - name
 - price
 - is_veg



■ Entity : Order

- order_id (PK)
- customer_id (FK)
- restaurant_id (FK)
- order_time
- status
- total_amount

■ Entity : OrderItem

- order_item_id (PK)
- order_id (FK)
- food_id (FK)
- quantity
- item_price

Step 3 – Relationships

■ a) Customer (1) — (N) Orders:

- One customer can place many orders.
- A customer may have 0 or many orders.

■ b) Restaurant (1) — (N) Orders

- One restaurant can receive many orders.
- Orders must belong to exactly one restaurant.

■ c) Restaurant (1) — (N) FoodItems

- One restaurant can have many menu items.
- Menu items must belong to exactly one restaurant.

■ d) Order (1) — (N) OrderItems

- One order can have many order line items.
- Every OrderItem must belong to exactly one Order.

■ e) FoodItem (1) — (N) OrderItems

- One food item can appear in many orders (via OrderItem).



2.5. Step-by-step: How to Draw in Draw.io (diagrams.net)

Step 1: Open Draw.io

- Go to: <https://app.diagrams.net/>
- Click Create New Diagram
 - Select Blank Diagram
 - Enter name: myfood-delivery-erd.png
 - Type : png
 - and Click on Create button

Step 2: Enable ERD / Database Shapes

- Top menu: View → Shapes...
- In the left pane, Click More Shapes:
 - Tick : Entity Relation
 - Click Apply
- Now, on the left sidebar, you'll see Entity shapes (table-like blocks).

Step 3: Create the Entities (Tables)

- We'll add one entity at a time.
- a) Add Customer
 - Drag an Entity shape to canvas.
 - Double-click → **title: Customer**
 - **Attributes:**
 - customer_id (PK)
 - name
 - phone
 - email
 - city

■ **b) Add Restaurant**

- Drag an Entity shape to canvas.
- Double-click → **title: Restaurant**
- **Attributes:**
 - restaurant_id (PK)
 - name
 - location
 - rating

■ **c) Add FoodItem**

- Drag an Entity shape to canvas.
- Double-click → **title: FoodItem**
- **Attributes:**
 - food_id (PK)
 - restaurant_id (FK)
 - name
 - price
 - is_veg

■ **d) Add Order**

- Drag an Entity shape to canvas.
- Double-click → **title: Order**
- **Attributes:**
 - order_id (PK)
 - customer_id (FK)
 - restaurant_id (FK)
 - order_time
 - status
 - total_amount



- **e) Add OrderItem**
 - Drag an Entity shape to canvas.
 - Double-click → **title: OrderItem**
 - **Attributes:**
 - order_item_id (PK)
 - order_id (FK)
 - food_id (FK)
 - quantity
 - item_price
- **Arrange them roughly like this:**
 - Customer: Top-left
 - Restaurant: Top-right
 - Order: Middle
 - OrderItem: Bottom-left
 - FoodItem: Bottom-right
- Now all entities + attributes + keys are on canvas.

Step 4: Draw Relationships with Connectors

- Use connectors from left panel
- Set Line Style → Crow's Foot for ERD connectors
- **Customer → Order (1 to Many)**
 - **Labels:**
 - Near Customer: 1
 - Near Order: N
- **Order → OrderItem (1 to Many)**
 - **Labels:**
 - Order: 1
 - OrderItem: N

- **Restaurant → FoodItem (1 to Many)**
 - **Labels:**
 - Restaurant: 1
 - FoodItem: N
- **FoodItem → OrderItem (1 to Many)**
 - **Labels:**
 - FoodItem: 1
 - OrderItem: N
- **Restaurant → Order (1 to Many)**
 - **Labels:**
 - Restaurant: 1
 - Order: N

Step 5: Clean Up & Export

- Right-click → Lock (prevents accidental movement)
- **Coloring (Optional):**
 - ✓ Give light colors to entities,
 - Customer = Blue
 - Restaurant = Orange
 - Order = Purple
 - OrderItem = Green
 - FoodItem = Red
- **Export Diagram:**
 - File → Export As → PNG / PDF / SVG
- **Save as:**
 - food-delivery-erd-v1.png



**Dandes
Academy**



3. Normalization & Normal Forms

- 3.1. What is Normalization?
- 3.2. Functional Dependencies (FDs)
- 3.3. First Normal Form (1NF)
- 3.4. Second Normal Form (2NF)
- 3.5. Third Normal Form (3NF)
- 3.6. BCNF (Boyce–Codd Normal Form)
- 3.7. SQL Denormalization
- 3.8. ASSIGNMENT – Food Delivery Normalization



3.1. What is Normalization?

- Normalization is a systematic process of organizing data in a database to:
 - Remove Redundancy
 - Avoid Anomalies
 - Ensure Consistency
 - Improve Efficiency
- It restructures large, cluttered tables into clean, smaller, well-organized tables.

A) Why Normalization is Needed?

1) Remove Redundancy

- Same information should not repeat across multiple rows
- Saves storage
- Improves data quality

2) Avoid Anomalies

- 3 Types of Anomalies
 - a) Insert Anomaly
 - b) Update Anomaly
 - c) Delete Anomaly

a) Insert Anomaly

- Problem:
 - You cannot insert some information because other unrelated information is required.
- Example:

- **Table: Students**

Student	Course	Instructor
A	Machine Learning	Sri

- Suppose you want to insert a new course:

Student	Course	Instructor
A	Machine Learning	Sri
—	Data Science	Vas

- But the table requires a Student, so you cannot insert a new course without a student.
- This is Insert Anomaly.

b) Update Anomaly

■ Problem:

- Data is stored in many rows → updating one place requires updating everywhere.
- If one row missed → inconsistent data.

■ Example:

▪ **Table: Students**

Student	Dept	DeptHead
A	CSE	Mr. Srinivas
B	CSE	Mr. Srinivas

- If the Dept Head changes from Mr. Srinivas → Mr. Ram
- You must update all rows where Dept = CSE.
- If one row is missed:

Student	Dept	DeptHead
A	CSE	Mr. Ram
B	CSE	Mr. Srinivas (Missed to update)

- This is Update Anomaly.



c) Delete Anomaly

- Problem:
 - Deleting one record unintentionally deletes important data.

- Example:

■ **Table: Students**

Student	Course
A	Machine Learning
B	Machine Learning

- Suppose A and B are the only two students taking Machine Learning.
- If both withdraw and you delete the rows:
 - Machine Learning course itself disappears from the database.
- This is Delete Anomaly.

Quick Summary

- **Insert Anomaly**
 - ✓ Cannot insert new info without extra info
- **Update Anomaly**
 - ✓ Same data in many rows must be updated everywhere
- **Delete Anomaly**
 - ✓ Deleting a row removes important data unintentionally

3) Improve Data Consistency

- Each fact is stored only once
- No contradictory values
- Reduces human mistakes during updates



4) Easy Maintenance

- Smaller, simpler tables
- Easy to update, search, and manage
- Better design clarity

5) Clear Relationships

- Normalization helps define:
 - Primary Keys
 - Foreign Keys
 - Correct Relational Structure

Goals of Normalization

Goal	Purpose
Eliminate redundancy	Avoid duplicate data
Reduce anomalies	Insert, update, delete issues solved
Ensure accuracy	Consistent and correct data
Improve structure	Proper relational design
Optimize storage	No unnecessary repetition

Normalization =

Organize data → Remove redundancy → Fix anomalies → Improve consistency



3.2. Functional Dependencies (FDs)

A) What is a Functional Dependency?

- Functional Dependency (FD) is a relationship between two attributes (columns) in a table.
- **Definition**
 - **Functional dependency $A \rightarrow B$ means:**
 - A uniquely determines B
 - or
 - For every value of A, there is only one corresponding value of B.
- **Interpretation:**
 - A = Determinant (left side)
 - B = Dependent attribute (right side)
- **Example:**
 - If there is a functional dependency from $\text{StudentID} \rightarrow \text{StudentName}$,
 - It means:
 - StudentID is the determinant
 - StudentName is functionally dependent on StudentID
 - For every StudentID, there is exactly one StudentName

C) Key Concepts in Functional Dependencies

- Core Concepts to understand.
 - 1) Determinant
 - 2) Dependent Attribute
 - 3) Full Functional Dependency
 - 4) Partial Dependency
 - 5) Transitive Dependency
 - 6) Candidate Keys
 - 7) Super Key
 - 8) Attribute Closure



1) Determinant

- Attribute(s) that determine another attribute.
- In FD: $A \rightarrow B$,
 - A is the determinant.
- Example:
 - In FD: $\text{StudentID} \rightarrow \text{StudentName}$,
 - StudentID is the determinant

2) Dependent Attribute

- Attribute that is uniquely Identified by determinant
- In FD: $A \rightarrow B$,
 - B is the dependent.
- Example:
 - In FD: $\text{StudentID} \rightarrow \text{StudentName}$,
 - StudentName is the dependent

3) Full Functional Dependency

- A dependency on the entire composite key and not on a part of it.
- Used in 2NF.
- Example:
 - $(\text{OrderID}, \text{ProductID}) \rightarrow \text{Quantity}$
 - Quantity depends on the full key.

4) Partial Dependency

- A dependency on part of a composite key, not the whole.
- Causes 2NF violation.
- Example:
 - $(\text{OrderID}, \text{ProductID}) \rightarrow \text{OrderDate}$
 - OrderDate depends only on OrderID, not ProductID $\rightarrow$ partial dependency.



5) Transitive Dependency

- A non-key attribute depends on another non-key attribute, not directly on the key.
- Occurs only when the dependency is indirect.
- Used in 3NF.
- **Example:**
 - order_id → Primary key
 - customer_id → non-key attribute
 - customer_city → non-key attribute
 - Functional Dependencies:
 - order_id → customer_id
 - customer_id → customer_city
 - So indirectly:
 - order_id → customer_city
 - This is a transitive dependency.

6) Candidate Keys

- A minimal set of attributes that uniquely identify a row.
- There can be more than one candidate key.
- **Example:**
 - Email
 - PhoneNumber
 - Both may uniquely identify Customer

7) Super Key

- Any combination of attributes that uniquely identify a record.
- Super Keys include candidate keys but can have extra attributes.
- **Example:**
 - (Email) → Unique → Candidate key.
 - (Email, Name) → Still unique → Superkey

8) Attribute Closure(X^+)

- **Attribute Closure of X =**
 - All attributes that X can determine using the given Functional Dependencies (FDs).
- **Think like this:**
 - If you know X, how many other attributes can you find?
 - That set = X^+ (Attribute Closure)
- **Example:**
 - Consider the following 2 tables and attributes:
 - **1) Orders Table**
 - order_id (PK)
 - customer_id (FK)
 - **2) Customers Table**
 - customer_id (PK)
 - customer_name
 - customer_city
 - **Functional Dependencies (FDs):**
 - order_id $\rightarrow$ customer_id
 - customer_id $\rightarrow$ customer_name
 - customer_id $\rightarrow$ customer_city
 - **Let's compute order_id⁺ (closure of order_id).**
 - **Step 0 — Start with order_id**
 - order_id⁺ = { order_id }
 - **Step 1 — Apply FD: order_id $\rightarrow$ customer_id**
 - Add customer_id to closure:
 - order_id⁺ = { order_id, customer_id }
 - **Step 2 — Apply FD: customer_id $\rightarrow$ customer_name**
 - Add customer_name:
 - order_id⁺ = { order_id, customer_id, customer_name }

- **Step 3 — Apply FD: $\text{customer_id} \rightarrow \text{customer_city}$**
 - Add customer_city :
 - $\text{order_id}^+ = \{ \text{order_id}, \text{customer_id}, \text{customer_name}, \text{customer_city} \}$
 - **Final Closure**
 - $\text{order_id}^+ = \{ \text{order_id}, \text{customer_id}, \text{customer_name}, \text{customer_city} \}$
 - **How to understand this**
 - If you know order_id , you automatically know customer_id
 - If you know customer_id , you get customer_name and customer_city
 - **So ultimately:**
 - order_id is the candidate key
 - order_id determines all other attributes in this table.
 - **Why Do We Use Attribute Closure?**
 - We use X^+ mainly for 3 reasons:
 - 1) To find candidate keys
 - 2) To check BCNF
 - 3) To verify dependencies
- 1) To find candidate keys**
- If X^+ contains all attributes in the table,
 - then X is a candidate key.
- 2) To check BCNF**
- BCNF requires that left side of FD must be a superkey.
 - So we compute the closure and check.
- 3) To verify dependencies**
- We check if one attribute determines another by computing closure.



D) How to Identify Functional Dependencies in Real Projects

- Functional Dependencies generally come from:
 - 1) Business Rules Dependent Attribute
 - 2) Domain Logic
 - 3) Understanding Real Data
 - 4) Existing System Knowledge

1. Business Rules

- Rules defined by how the company works.
- **Example:**
 - Each order is placed by exactly one customer.
 - $\text{order_id} \rightarrow \text{customer_id}$

2. Domain Logic

- Rules based on how the domain behaves.
- **Example:**
 - Each food item belongs to exactly one restaurant.
 - $\text{food_id} \rightarrow \text{restaurant_id}$

3. Understanding Real Data

- Look at actual data — what is unique?
- **Example:**
 - A customer's phone number is unique.
 - $\text{phone} \rightarrow \text{customer_id}$

4. Existing System Knowledge

- Understanding how the current system stores relationships.
- **Example:**
 - In the OrderItem table:
 - $(\text{order_id}, \text{food_id})$ uniquely determines quantity and price at order time.
 - $(\text{order_id}, \text{food_id}) \rightarrow \text{quantity}, \text{item_price}$

3.3. First Normal Form (1NF)

- Table is in First Normal Form (1NF) when
 - Data is stored in atomic form with no repeating groups.

1NF = Atomic values + No repeating groups

- This means:
 - a) All values are atomic (no multiple values in one cell)
 - b) No repeating groups / no repeating columns

Example: Check whether the following table is in 1NF

- Table: myorders

order_id	product_names
101	Pen, Notebook
102	Laptop, Charger, Mouse

- Why this violates 1NF?
 - **product_names** column contains multiple values separated by commas.
 - Values are **not atomic**.
 - It contains **repeating groups**.

Therefore, the table is NOT in 1NF.

Converting the Table to 1NF

- Table: myorders ()

order_id	product_name
101	Pen
101	Notebook
102	Laptop
102	Charger
102	Mouse



3.4. Second Normal Form (2NF)

- A table is in Second Normal Form (2NF) when:
 - a) It meets all rules of 1NF
 - b) It has NO Partial Dependency on any candidate key

2NF = 1NF + No Partial Dependencies

What is a Partial Dependency?

- A partial dependency occurs when:
 - A non-key attribute depends on PART of a composite key,
 - instead of depending on the whole composite key.
- This only happens when the primary key has more than one column.
- If the table has a single-column primary key →
 - Partial dependency cannot occur → automatically in 2NF (if in 1NF).

Check whether the following table is in 2NF?

- Table: order_items_raw

order_id	product_id	product_name	customer_id
101	P01	Pen	C001
101	P02	Notebook	C001
102	P01	Pen	C002

- Step 1: Check 1NF
 - It is already in 1NF (all values are atomic)
- Step 2: Identify the Primary Key
 - order_id alone is not unique → An order can have multiple products.
 - product_id alone is not unique → Same product can appear in different orders.
 - Therefore, the composite primary key is: (**Order_ID , Product_ID**)



■ Step 3: Check for Partial Dependencies

- product_name → depends only on product_id
- customer_name → depends only on order_id
- Both attributes do NOT depend on the full composite key.
- So partial dependencies exist.
 - Therefore, this table is NOT in 2NF.

Converting the Table to 2NF (Removing Partial Dependencies)

1) Orders Table → attributes that depend only on order_id

order_id	customer_id
101	C001
102	C002

2) Products Table → attributes that depend only on Product_ID

product_id	product_name
P01	Pen
P02	Notebook

3) Order_Items Table → intersection of Order and Product

order_id	product_id
101	P01
101	P02
102	P01

- Now all non-key attributes depend on the FULL key.
 - No partial dependencies
 - Table is in 2NF

2NF = 1NF + No Partial Dependencies



3.5. Third Normal Form (3NF)

- A table is in Third Normal Form (3NF) when:
 - a) It is already in 2NF
 - b) There is NO Transitive Dependency

3NF = 2NF + No Transitive Dependencies.

What is a Transitive Dependency?

- A transitive dependency occurs when:
 - A non-key attribute depends on another non-key attribute,
 - instead of depending directly on the primary key.
- Example pattern:
 - **Primary Key → Non-key A → Non-key B**
- Non-key B indirectly depends on the primary key.
- This violates 3NF.

Check whether the following table is in 3NF?

- Table: order_items_raw

sid	sname	cid	cname	instructor	inst_phone
201	aaa	C-01	Java	Sri	55555
202	bbb	C-02	Python	Vas	99999
203	ccc	C-01	Java	Sri	55555
201	aaa	C-02	Python	Vas	99999

- Step 1: Check 1NF
 - All values are atomic.
 - Table is in 1NF.

■ Step 2: Check 2NF

- Primary Key = (sid , cid)
- Reason: A student can take multiple courses.
- Check Partial Dependencies (non-key attributes depending on part of key):
 - sname - depends only on sid
 - cname - depends only on cid
 - instructor - depends only on cid
 - inst_phone - depends only on instructor (not on full key)
- These are partial dependencies.
- Table is NOT in 2NF.

Converting the Table to 2NF (Removing Partial Dependencies)

1) mystudents (Attributes that depend only on sid)

sid	sname
201	aaa
202	bbb
203	ccc

2) mycourses (Attributes that depend only on cid)

cid	cname	instructor	inst_phone
C01	Java	Sri	55555
C02	Python	Vas	99999

3) enrollments (Intersection of Student and Course)

sid	cid
201	C-01
202	C-02
203	C-01
201	C-02

■ Now - No non-key attribute depends on part of a composite key.

- All partial dependencies are removed.
- Table is in 2NF.

■ **Step 3: Check for Transitive Dependency (3NF)**

Recall: 3NF = 2NF + No Transitive Dependencies.

- Look inside mycourses table:
 - cid → instructor
 - instructor → inst_phone
- So indirectly:
 - cid → inst_phone
- This is a TRUE Transitive Dependency.
 - **So mycourses is NOT in 3NF.**

Convert to 3NF (Remove Transitive Dependency)

4) instructors table:

instructor	inst_phone
Sri	55555
Vas	99999

Updated mycourses (remove instructor's phone)

cid	cname	instructor
C01	Java	Sri
C02	Python	Vas

Final Result

- mystudents → in 3NF
 - enrollments → in 3NF
 - mycourses → now in 3NF after removing transitive dependency
 - instructors → in 3NF
- All tables are now fully normalized.



3.6. BCNF (Boyce–Codd Normal Form)

- A table is in BCNF when:
 - a) It is already in 3NF
 - b) For every Functional Dependency ($A \rightarrow B$),
 - A must be a Super Key (Candidate Key)
 - Meaning:
 - **LEFT side (determinant) of every FD must be a KEY.**

BCNF = 3NF + Every determinant must be a key

Check whether the following table is in BCNF?

- **Table: mycourses**

course	instructor	InstructorDept	room
ML	A123	AI	R1
DL	A124	AI	R1
Java	A125	IT	R2
Python	A126	CS	R3

- **Step 1: Check 1NF**
 - All values are atomic (no repeating groups).
 - Table is in 1NF.
- **Step 2: Check 2NF**
 - Primary Key = course (single-column key)
 - Since the primary key is not composite,
 - No partial dependency possible.
 - The table is in 2NF.

■ **Step 3: Check 3NF**

- **Functional Dependencies:**

- FD1: Course → Instructor, Room
- FD2: Instructor → InstructorDept (each instructor belongs to one dept)
- FD3: Room → Instructor

- **Now check for transitive dependency:**

- Course → Instructor
- Instructor → InstructorDept
- So: Course → InstructorDept (via Instructor)
- This is a transitive dependency.
 - **So mycourses is NOT in 3NF.**

Convert to 3NF (Remove Transitive Dependency)

■ **Table: mycourses**

course	instructor	room
ML	A123	R1
DL	A124	R1
Java	A125	R2
Python	A126	R3

2) instructor_dept

instructor	InstructorDept
A123	AI
A124	AI
A125	IT
A126	CS

Now there are:

- No partial dependencies
- No transitive dependencies
- Both tables are in 3NF

■ **Step 4: Check BCNF**

✓ **Now examine mycourses table**

- Table: mycourses

course	instructor	room
ML	A123	R1
DL	A124	R1
Java	A125	R2
Python	A126	R3

- **BCNF rule:**
 - For every $A \rightarrow B$, A must be a KEY.
- **Functional Dependencies here:**
 - FD1: Course $\rightarrow$ Instructor, Room
 - (Course is a key $\rightarrow$ ✓ OK for BCNF)
 - FD2: Room $\rightarrow$ Instructor
 - **Room is NOT a key of this table.**
 - But Room determines Instructor.
- **This violates the BCNF rule:**
 - **Determinant (Room) is not a key.**
 - **Table is NOT in BCNF.**

Convert to BCNF

- We decompose based on the violating FD: Room $\rightarrow$ Instructor

1) room_instructor (BCNF)

room	instructor
R1	A123
R1	A124
R2	A125
R3	A126

- Here Room is the key $\rightarrow$ ✓ BCNF

2) course_room (BCNF)

course	room
ML	R1
DL	R1
Java	R2
Python	R3

- Here Course is the key $\rightarrow$ ✓ BCNF

We already have instructor_dept from 3NF step:

instructor	InstructorDept
A123	AI
A124	AI
A125	IT
A126	CS



Final Result:

- After BCNF decomposition, we have three tables:
 - 1) course_room
 - 2) room_instructor
 - 3) instructor_dept
- All determinants (Course, Room, Instructor) are keys.
- Table is in BCNF

3.7. SQL Denormalization

- Denormalization is the process of combining tables to reduce the complexity of queries.
- It intentionally introduces data redundancy to improve read performance.
- In simple words:
 - You trade **storage space** for **speed**.

Why Denormalization is Needed?

- a) Reduce JOINS
 - Multiple joins slow down SELECT queries.
- b) Improve Query Performance
 - Fewer joins → Faster reads
- c) Faster Data Retrieval
 - Best for dashboards, reports, analytics (OLAP).

Example – Before Denormalization

■ **Table: Customers**

cid	cname	email
C1	Sri	sri@da.com
C2	Vas	vas@da.com

■ **Table: Orders**

order_id	cid	order_date
101	C1	2025-12-01
102	C2	2025-12-02

■ **Table: Order_Items**

order_item_id	order_id	product	quantity
1	101	Pen	1
2	101	Notebook	1
3	102	Pen	5

After Denormalization

- Combine Customers + Orders + Order_Items:

order_id	cname	email	product	quantity
101	Sri	sri@da.com	Pen	1
101	Sri	sri@da.com	Notebook	1
102	Vas	vas@da.com	Pen	5

■ **Now**

- No JOINS needed
- Faster read queries
- Simpler reports



Denormalization – Pros & Cons

■ Advantages:

- No JOINS needed
- Faster Query Execution
- Easier Data Retrieval
- Best for read-heavy workloads (analytics/reporting)

■ Disadvantages:

- Data Redundancy (duplicate data)
- Higher Storage Usage
- Update Anomalies
- Risk of Data Inconsistency

Denormalization – Pros & Cons

Feature	Normalization	Denormalization
Purpose	Reduce redundancy, improve integrity	Improve query performance
Data Redundancy	Very low	Introduced intentionally
Performance	Slower reads, faster writes	Faster reads, slower writes
Query Complexity	JOINS required	Fewer joins, simpler queries
Number of Tables	More tables (split data)	Fewer tables (merged data)
Use Case	OLTP: Banking, E-commerce	OLAP: Analytics, Reporting

Normalization = Accuracy + Integrity

Denormalization = Speed + Fewer Joins



3.8. ASSIGNMENT – Food Delivery Normalization

- **Domain:** Swiggy / Zomato style orders
- **Context:**
 - ✓ A food delivery company stores order details in a single wide table.
 - ✓ Your tasks:
 - Identify issues in the schema
 - Normalize 1NF → 2NF → 3NF → BCNF
 - Produce the final clean schema + ERD
- **Difficulty:**
 - ✓ Medium-High | Real-World Case Study

You are given the following UNNORMALIZED table:

- **Table: food_orders**

order_id	cname	restaurant_name	restaurant_area	item_names	delivery_partner	partner_zone
101	Srinivas	KFC	BTM	Burger,Fries,Coke	Rider1	Zone-A
102	Dandes	Domino's	HSR	Pizza,Garlic Bread	Rider2	Zone-B
103	Srinivas	KFC	BTM	Burger,Coke	Rider1	Zone-A

SECTION-1: Understanding the Raw Data (10 Marks)

- Q1) Identify any 3 problems with the above schema (anomalies / redundancy / 1NF violations / repeating info). **(3 marks)**
- Q2) If items are split into individual rows, what would be the composite primary key? **(2 marks)**
- Q3) Write all Functional Dependencies (FDs) visible in the table. **(5 marks)**



SECTION-2: First Normal Form (1NF) (10 Marks)

Q4) Convert the above table into 1NF by removing multivalued attributes. **(8 marks)**
Provide the 1NF table.

Q5) Explain why item_names violates 1NF. **(2 marks)**

SECTION-3: Second Normal Form (2NF) (10 Marks)

- Assume the 1NF table uses a composite primary key: (order_id, item_name)

Q6) Identify all partial dependencies that violate 2NF. **(3 marks)**

Q7) Decompose 1NF into multiple tables such that the design satisfies 2NF.
Provide only table names + attributes (no sample data). **(7 marks)**

SECTION-4: Third Normal Form (3NF) (15 Marks)

- After 2NF decomposition,
 - ✓ one table will still contain non-key attributes that depend on other non-key attributes.

Q8) Identify the transitive dependencies present in that table. **(5 marks)**

Q9) Decompose the table further into 3NF-compliant tables.
Provide table names + attributes. **(10 marks)**



SECTION-5: Boyce-Codd Normal Form (BCNF) (20 Marks)

- You will observe an FD like: $X \rightarrow Y$
 - ✓ Where X is NOT a superkey.

Q10) Explain why such an FD causes a BCNF violation. **(5 marks)**

Q11) Decompose the affected tables to achieve BCNF.

Provide the final BCNF table names + attributes. **(10 marks)**

Q12) Explain why BCNF is stricter than 3NF (definition + example). **(5 marks)**

SECTION-6: Final Output – ERD (15 Marks)

Q13) Draw the ERD of your FINAL BCNF schema. **(15 marks)**

Must include:

- all entities
- PK/FK relationships
- cardinalities

SECTION-7: Short Answers (20 Marks)

Q14) Define: **(8 marks)**

- a) Partial Dependency
- b) Transitive Dependency
- c) Candidate Key
- d) Surrogate Key

Q15) Give ONE real-world example for: **(12 marks)**

- 1NF violation
- 2NF violation
- 3NF violation
- BCNF violation

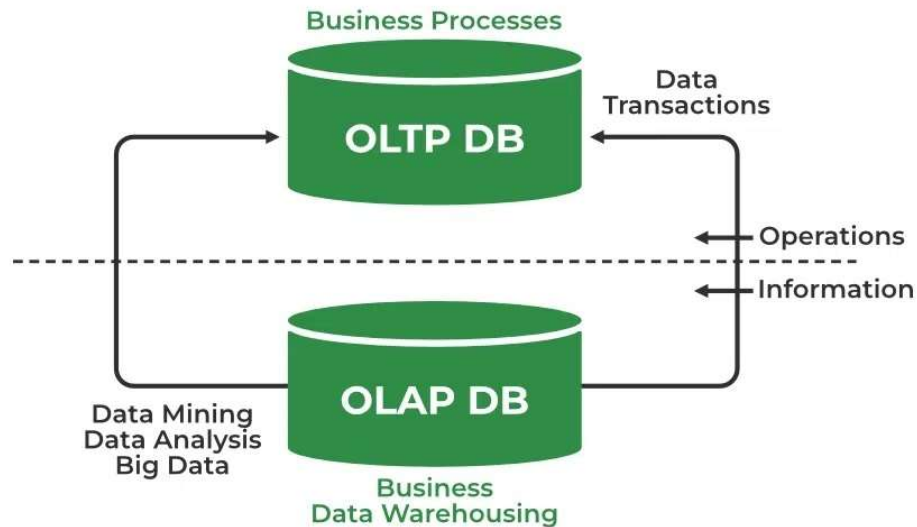


4. Data Warehouse Design

- 4.1. Why Data Warehouses?
- 4.2. Facts & Dimensions
- 4.3. Star Schema
- 4.4. Snowflake Schema
- 4.5. Star Schema vs Snowflake Schema
- 4.6. Mini Case Study — Star Schema
- 4.7. Mini Case Study — Snowflake Schema

4.1. Why Data Warehouses?

OLTP vs OLAP — Quick Recap



OLTP Databases:	OLAP Databases
Represents Business Processes	Represents Business Data Warehousing
Handles day-to-day transactions	Used for: Data Analysis, Data Mining and Big Data
Highly normalized	Denormalized / Star Schema
Used by applications	Stores historical + aggregated data
Focus: Real-time Operations	Focus: Information & Insights
Examples: Place order Update payment Add customer Book ticket	Examples: Monthly sales report Revenue by city / product Customer trends over 5 years Top-selling products analysis
OLTP systems RUN the business.	OLAP systems ANALYZE the business.



Why Data Warehouses Exist?

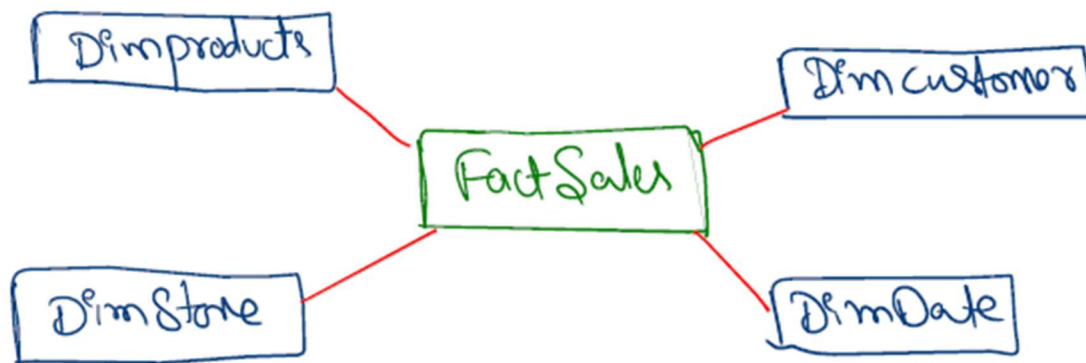
- Companies build a Data Warehouse because:
 - 1) Application DB cannot handle heavy analytical queries
 - 2) Business teams need reports, dashboards, insights
 - 3) DW stores long-term historical data (5–10+ years)
 - 4) DW is read-optimized, not write-optimized
 - 5) DW integrates data from many systems:
 - CRM
 - Billing
 - E-commerce
 - Marketing
 - Support

Key Features of a Data Warehouse

- Read-optimized (fast queries)
- Stores historical data
- Subject-oriented (Sales, Customer, Product)
- Integrated (combines data from multiple sources)
- Non-volatile (data doesn't change frequently)

4.2. Facts & Dimensions

- At the core of every Data Warehouse are two fundamental building blocks:
 - ✓ Fact Tables
 - → Stores numerical data (what happened)
 - ✓ Dimension Tables
 - → Stores descriptive information (details about what happened)



- Facts & Dimensions form the HEART of Data Warehousing.
- If you understand Facts & Dimensions properly → then you can learn the following easily.
 - ✓ Star Schema
 - ✓ Snowflake Schema
 - ✓ Different Fact Table types
 - ✓ Slowly Changing Dimensions (SCD)
- Next We will explore the following
 - ✓ A) Fact Tables
 - ✓ B) Dimension Tables
 - ✓ C) Granularity
 - ✓ D) Surrogate Keys



A) Fact Tables

- **What is a Fact Table?**
 - ✓ A Fact Table stores the **measurable, numerical data** of the business.
 - ✓ It answers the question: “**What happened?**”
- **Examples:**
 - ✓ How many items sold?
 - ✓ How much revenue?
 - ✓ What was the discount?
 - ✓ How many orders?
- **Characteristics:**
 - ✓ Contains numeric measures
 - ✓ Contains foreign keys to dimensions
 - ✓ Very large (millions/billions of rows)
 - ✓ Granular (lowest level of detail)
- **Example: FactSales**

Table Structure

Column Name	Type	Meaning
sale_id	Identifier	Unique ID for each sale
date_key	FK	Links to Date dimension
product_key	FK	Links to Product dimension
customer_key	FK	Links to Customer dimension
store_key	FK	Links to Store dimension
quantity	Measure	Number of items sold
amount	Measure	Total sales amount

sale_id	date_key	product_key	customer_key	store_key	quantity	amount
1	20250101	10	501	20	2	1000
2	20250101	12	502	20	1	500



- ✓ Each row represents one sales transaction (one business event).
- ✓ **Row 1 means:**
 - On **2024-01-01**
 - Customer **501**
 - Bought Product **10**
 - From Store **20**
 - Quantity sold = **2 items**
 - Total amount = **1000**
- ✓ **This is ONE sales event**

B) Dimension Tables

- What is a Dimension Table?
 - ✓ A Dimension Table stores **Descriptive/Text information** about business entities.
 - ✓ It answers questions like: **Who, What, Where, When, and How?**
- **Examples:**
 - ✓ Who is the customer?
 - ✓ What is the product?
 - ✓ Where is the store?
 - ✓ When did the sale happen?
- **Characteristics:**
 - ✓ Textual attributes (names, categories, locations)
 - ✓ Small in size compared to facts
 - ✓ Joined to fact table using surrogate keys



■ Example: DimProduct

product_key	product_name	category	brand
10	iPhone 16	Mobile	Apple
12	Samsung M34	Mobile	Samsung

Explanation:

- Each row represents one product.
 - ✓ **product_key** → Surrogate key (used to join with Fact table)
 - ✓ **product_name** → Actual product sold
 - ✓ **category** → High-level grouping (Mobile, Laptop, etc.)
 - ✓ **brand** → Manufacturer (Apple, Samsung)
- When product_key = 10 appears in the Fact table,
 - ✓ it means iPhone 15 was sold.

■ Example: DimCustomer

customer_key	customer_name	city	state
501	Srinivas	Bangalore	KA
502	Vas	Hyderabad	TS

Explanation:

- Each row represents one customer.
 - ✓ **customer_key** → Surrogate key for customer
 - ✓ **customer_name** → Customer identity
 - ✓ **city, state** → Location details
- When customer_key = 501 appears in FactSales,
 - ✓ it means Srinivas from Bangalore made the purchase.



▪ Example: DimDate

date_key	date	month	year
20250101	2025-01-01	Jan	2025
20250101	2025-01-01	Jan	2025

Explanation:

- Each row represents one calendar date.
 - ✓ **date_key** → Surrogate key (YYYYMMDD format)
 - ✓ **date** → Actual calendar date
 - ✓ **month, year** → Time attributes for grouping

▪ From FactSales table

sale_id	date_key	product_key	customer_key	store_key	quantity	amount
1	20250101	10	501	20	2	1000
2	20250101	12	502	20	1	500

▪ Business Questions & Answers

Q1) WHEN did the sale happen?

- ✓ From DimDate: date_key = 20240101 → 01-Jan-2024

Q2) WHO bought the product?

- ✓ From DimCustomer: customer_key = 501 → Srinivas, Bangalore

Q3) WHAT product was bought?

- ✓ From DimProduct: product_key = 10 → iPhone 15 (Mobile, Apple)

Q4) HOW MANY items were bought?

- ✓ From Fact table: quantity = 2



Q5) HOW MUCH was the total amount?

- ✓ From Fact table: amount = 1000

Q6) WHERE did the sale happen? (if store dimension exists)

- ✓ From DimStore: store_key → Bangalore Store

■ **Final Combined Business Meaning**

- ✓ On **01-Jan-2025**,
- ✓ **Srinivas** from **Bangalore**
- ✓ bought **2 units** of **iPhone 15**,
- ✓ worth **₹1000**.

C) Granularity

■ **What is Granularity?**

- ✓ Granularity refers to the **level of detail** stored in a Fact Table.

■ **Examples of granularity:**

- ✓ One row per sale (most detailed)
- ✓ One row per order
- ✓ One row per day per store
- ✓ One row per month (least detailed)

■ **Choosing granularity:**

- ✓ Lowest granularity → more rows → more detail
- ✓ Highest granularity → fewer rows → summary

Example 1 — One row per sale:

■ **Customer buys 2 products in one order → 2 rows in fact table**

■ This is:

- ✓ Lowest granularity
- ✓ Most detailed
- ✓ Best design choice

■ You can answer:

- ✓ Product-level sales
- ✓ Customer-product behavior
- ✓ Item-level discounts

Example 2 — One row per day per store

- **Store 20 | Date 2025-01-01 | Total Sales = 1,00,000**
- This is:
 - ✓ Higher granularity
 - ✓ Aggregated data
 - ✓ Less detailed
- You cannot answer:
 - ✓ Which product sold?
 - ✓ Which customer bought?
 - ✓ Item-level discount?

Best Practice: Always choose the Lowest granularity during data warehouse design.

D) Surrogate Keys

- **What is a Surrogate Key?**
 - ✓ A Surrogate Key is a system-generated integer key used to uniquely identify records in dimension tables and to join them with fact tables.
- **Why surrogate keys?**
 - ✓ Natural keys (product_id, customer_id) may change
 - ✓ Surrogate keys NEVER change
 - ✓ Better join performance (int join is faster than text)
 - ✓ Essential for SCD Type 2 (versioning data)
- **Example:**
 - ✓ **In DimProduct:**
 - product_key (Surrogate Key) → 10
 - product_id from OLTP system → P1009
 - Data Warehouses do NOT use OLTP natural IDs for joins.
- **Key Points:**
 - ✓ Natural keys belong to OLTP systems.
 - ✓ Surrogate keys belong to Data Warehouses.



Summary

- Facts store numeric measures and foreign keys
 - Dimensions store descriptive attributes
 - Fact tables are very large (millions to billions of rows)
 - Dimension tables are smaller and descriptive
 - Granularity defines the level of detail in fact tables
 - Surrogate keys are stable, numeric keys used for joins
-
- **Together, Facts and Dimensions form the foundation of Star and Snowflake schemas.**

4.3. Star Schema

What is a Star Schema?

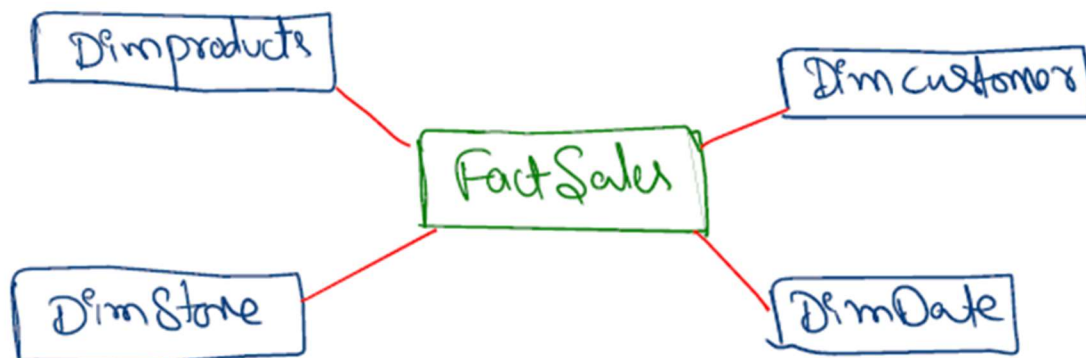
- Star Schema is a Data Warehouse design where:
 - ✓ One central table acts as the FACT TABLE
 - ✓ Multiple surrounding tables act as DIMENSION TABLES
 - ✓ Each dimension connects directly to the fact table
 - ✓ Structure resembles a STAR shape

Why use Star Schema?

- Star Schema is the MOST widely used Data Warehouse design in the industry.
 - ✓ Simple and easy to understand
 - ✓ Optimized for analytical queries
 - ✓ Works best with BI tools (Power BI, Tableau, Looker)
 - ✓ Fewer joins → better query performance
 - ✓ Ideal for reporting and dashboards

Star Schema Structure (General Format)

- Fact Table sits in the middle.
- Dimensions Tables sit around it like points of a star.





Retail Example — Sales Analytics

■ Table: FactSales

sale_id	date_key	product_key	customer_key	store_key	quantity	amount
1	20250101	10	501	20	2	1000
2	20250101	12	502	20	1	500

■ Table: DimProducts

product_key	product_name	category	brand
10	iPhone 16	Mobile	Apple
12	Samsung M34	Mobile	Samsung

■ Table: DimCustomer

customer_key	customer_name	city	state
501	Srinivas	Bangalore	KA
502	Vas	Hyderabad	TS

■ Table: DimDate

date_key	date	month	year
20250101	2025-01-01	Jan	2025
20250101	2025-01-01	Jan	2025

■ Table: DimStore

store_key	store_name	location
20	Store A	Bangalore



4.4. Snowflake Schema

What is a Snowflake Schema?

- Snowflake Schema is an extension of the Star Schema where:
 - ✓ Dimension tables are normalized
 - ✓ A dimension can be split into multiple sub-dimensions
 - ✓ Structure resembles a snowflake with branching patterns
- ✓ Fact Table still sits at the center,
 - but dimension tables are broken into multiple related tables.

Why Snowflake?

- Snowflake Schema is the Preferred because of the following reasons.
 - ✓ Reduces duplicate data in dimensions
 - ✓ Maintains clean hierarchical relationships
 - ✓ Improves data consistency and quality
 - ✓ Saves storage for very large dimensions

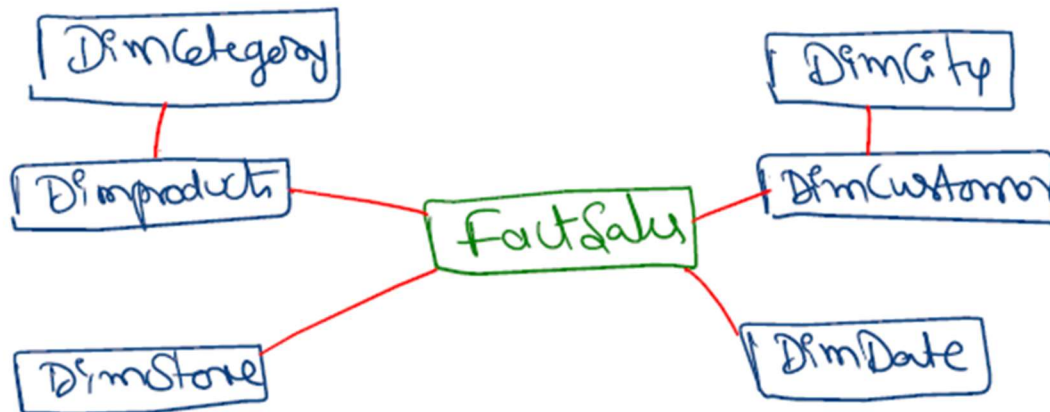
When to use Snowflake?

- Use Snowflake Schema when:
 - ✓ Very large dimension tables
 - ✓ Dimensions with clear hierarchies (City → State → Country)
 - ✓ When storage is expensive
 - ✓ When strict normalization is required

Star vs Snowflake

- Star Schema
 - ✓ → Dimensions are denormalized (flat)
- Snowflake Schema
 - ✓ → Dimensions are normalized (multiple tables)

Retail Example — Same as Star Schema but Dimensions are Normalized



■ **Table: FactSales**

sale_id	date_key	product_key	customer_key	store_key	quantity	amount
1	20250101	10	501	20	2	1000
2	20250101	12	502	20	1	500
3	20250102	10	503	21	1	500

■ **Table: DimProducts**

product_key	product_name	category_key	brand
10	iPhone 16	101	Apple
12	Samsung M34	101	Samsung
20	Dell Inspiron	102	Dell

■ **Table: DimCategory**

category_key	category_name
101	Mobile
102	Laptop



■ **Table: DimCustomer**

customer_key	customer_name	city_key
501	Srinivas	55
502	Vas	56
503	Sri	55

■ **Table: DimCity**

city_key	city_name	state
55	Bangalore	KA
56	Hyderabad	TS

■ **Table: DimDate**

date_key	date	month	year
20250101	2025-01-01	Jan	2025
20250101	2025-01-01	Jan	2025

■ **Table: DimStore**

store_key	store_name	location
20	Store-Bangalore	Bangalore
21	Store-Hyderabad	Hyderabad



4.5. Star Schema vs Snowflake Schema

Feature	Star Schema	Snowflake Schema
Dimension	Structure Denormalized (flat)	Normalized (split into tables)
Joins	Fewer joins	More joins
Query Performance	Faster	Slightly slower
Storage Usage	More (Data Duplication)	Less (Reduced Duplication)
Data Consistency	Good	Better
Complexity	Simple	More complex
Ease of Understanding	Very easy	Moderate
BI Tool Support	Excellent	Good
Hierarchy Handling	Stored in one table	Stored across tables
Best Used When	Performance & simplicity needed	Large dimensions & storage matters
Example Structure	FactSales ↳ DimProduct ↳ DimCustomer ↳ DimDate ↳ DimStore	FactSales ↳ DimProduct → DimCategory ↳ DimCustomer → DimCity ↳ DimDate ↳ DimStore



4.6. Mini Case Study — Food Delivery Data Warehouse (STAR)

What is this Case Study about?

- We are designing a small Data Warehouse for a Food Delivery company (like Swiggy / Zomato / Uber Eats).
- Goal:
 - ✓ To analyze food orders:
 - Which restaurants generate most revenue?
 - Which food items sell the most?
 - Which cities have higher order volume?
 - Who are the top customers?
 - How is monthly revenue trending?

Step 1 – Understand the Business

- **Business:**
 - ✓ Food Delivery App connects:
 - Customers
 - Restaurants
 - Delivery Partners
 - Food Items
 - Orders & Payments
 - ✓ Users place orders → Restaurants prepare → Delivery partners deliver.
 - ✓ We want to do ANALYTICS, not transactions.

Step 2 – Understand Source (OLTP)

- Typical OLTP tables:
 - ✓ **customers**(customer_id, name, city, state)
 - ✓ **orders**(order_id, order_date, customer_id, restaurant_id, total_amount)
 - ✓ **order_items**(order_item_id, order_id, food_item_id, quantity, line_amount)
 - ✓ **restaurants**(restaurant_id, restaurant_name, cuisine_type, city)
 - ✓ **food_items**(food_item_id, food_name, category, price)
- These are NOT copied as-is into Data Warehouse
- We convert them into FACT + DIMENSIONS.



Step 3 – Identify Business Questions

- Before modeling, we ask: "What do they want to see?"
- **Example Questions:**
 - ✓ 1) Total revenue per month?
 - ✓ 2) Top 10 restaurants by revenue?
 - ✓ 3) Most ordered food items?
 - ✓ 4) City-wise order trends?
 - ✓ 5) Top customers by spend?
- **These tell us:**
 - ✓ Main process = FOOD ORDERS
 - ✓ Dimensions = Date, Customer, Restaurant, Food Item

Step 4 – Choose the Business Process & Grain

- **Business Process:**
 - ✓ FOOD SALES
 - ✓ So our fact table will be: FactFoodSales
- **Grain (very important):**
 - ✓ Grain = 1 row per food item per order
 - ✓ **Example:**
 - **If one order has:**
 - Biryani
 - Coke
 - Fries
 - → **FactFoodSales has 3 rows.**
 - This gives maximum analysis flexibility.

Step 5 – Identify Dimensions

■ **Chosen Dimensions:**

- ✓ DimDate → time analysis
- ✓ DimCustomer → customer & location
- ✓ DimRestaurant → restaurant & cuisine
- ✓ DimFoodItem → food & category

(Optional later):

- ✓ • DimDeliveryPartner
- ✓ • DimPaymentMode



Step 6 – Design Dimension Tables

■ **Table: DimDate**

date_key	date	day_name	month	quarter	year
20260101	2026-01-01	Thursday	Jan	Q1	2026

■ **Table: DimFoodItem**

food_key	food_id	food_name	category
10	F101	Biryani	Main Dish
11	F102	Burger	Snack



■ **Table: DimCustomer**

customer_key	customer_id	customer_name	city	state
501	C1	Srinivas	Bangalore	KA

■ **Table: DimRestaurant**

restaurant_key	restaurant_id	restaurant_name	cuisine	city
20	R1	Meghana Foods	Andhra	Bangalore

Step 7 – Design Fact Table (FactFoodSales)

■ **Purpose:**

- ✓ Stores each food item sold in each order.

■ **Columns:**

- ✓ sale_id
- ✓ date_key
- ✓ food_key
- ✓ customer_key
- ✓ restaurant_key
- ✓ quantity
- ✓ amount

■ **Each row = 1 food item in 1 order**

■ **Table: FactFoodSales**

sale_id	date_key	food_key	customer_key	restaurant_key	quantity	amount
1	20260101	10	501	20	2	500
2	20260101	11	501	20	2	120



Step 8 – Star Schema Diagram

- ✓ **Center:**
 - FactFoodSales
- ✓ **Surrounding dimensions:**
 - DimDate, DimCustomer, DimRestaurant, DimFoodItem



SQL Design — Food Delivery Star Schema

-- Food Delivery Data Warehouse (Star Schema)
-- DimDate, DimFoodItem, DimCustomer, DimRestaurant, FactFoodSales

-- **DROP TABLES (optional, for re-run)**

-- DROP TABLE FactFoodSales;
-- DROP TABLE DimDate;
-- DROP TABLE DimFoodItem;
-- DROP TABLE DimCustomer;
-- DROP TABLE DimRestaurant;



-- 1) Dimension Tables --

-- DimDate

```
CREATE TABLE DimDate (  
    date_key INT PRIMARY KEY,    -- e.g., 20240101  
    full_date DATE,  
    day_name VARCHAR(10),  
    month_name VARCHAR(10),  
    quarter VARCHAR(2),  
    year INT  
);
```

-- DimFoodItem

```
CREATE TABLE DimFoodItem (  
    food_key INT PRIMARY KEY,    -- surrogate key  
    food_id VARCHAR(20),        -- OLTP food item id  
    food_name VARCHAR(100),  
    category VARCHAR(50)        -- Main Dish / Snack / Beverage / Dessert  
);
```

-- DimCustomer

```
CREATE TABLE DimCustomer (  
    customer_key INT PRIMARY KEY,  
    customer_id VARCHAR(20),    -- OLTP customer id  
    customer_name VARCHAR(100),  
    city VARCHAR(50),  
    state VARCHAR(50)  
);
```

-- DimRestaurant

```
CREATE TABLE DimRestaurant (  
    restaurant_key INT PRIMARY KEY,  
    restaurant_id VARCHAR(20),  -- OLTP restaurant id  
    restaurant_name VARCHAR(100),  
    cuisine VARCHAR(50),  
    city VARCHAR(50)  
);
```



-- 2) Fact Table --

-- FactFoodSales (grain = 1 row per food item per order)

```
CREATE TABLE FactFoodSales (  
    sale_id      INT PRIMARY KEY,  
    date_key     INT,  
    food_key     INT,  
    customer_key INT,  
    restaurant_key INT,  
    quantity     INT,  
    amount       DECIMAL(12,2), -- line-level revenue  
  
    FOREIGN KEY (date_key)   REFERENCES DimDate(date_key),  
    FOREIGN KEY (food_key)   REFERENCES DimFoodItem(food_key),  
    FOREIGN KEY (customer_key) REFERENCES DimCustomer(customer_key),  
    FOREIGN KEY (restaurant_key) REFERENCES DimRestaurant(restaurant_key)  
);
```

-- 3) Insert Dim Data --

-- DimDate

```
INSERT INTO DimDate (date_key, full_date, day_name, month_name, quarter, year) VALUES  
(20250101, '2025-01-01', 'Monday', 'Jan', 'Q1', 2024),  
(20250102, '2025-01-02', 'Tuesday', 'Jan', 'Q1', 2024),  
(20250201, '2025-02-01', 'Thursday', 'Feb', 'Q1', 2024),  
(20250210, '2025-02-10', 'Saturday', 'Feb', 'Q1', 2024),  
(20250305, '2025-03-05', 'Tuesday', 'Mar', 'Q1', 2024);
```

-- DimFoodItem

```
INSERT INTO DimFoodItem (food_key, food_id, food_name, category) VALUES  
(10, 'F101', 'Chicken Biryani', 'Main Dish'),  
(11, 'F102', 'Veg Burger', 'Snack'),  
(12, 'F103', 'Coke', 'Beverage'),  
(13, 'F104', 'Gulab Jamun', 'Dessert');
```



-- DimCustomer

```
INSERT INTO DimCustomer (customer_key, customer_id, customer_name, city, state) VALUES
(501, 'C1', Srinivas, 'Bangalore', 'KA'),
(502, 'C2', Vas, 'Hyderabad', 'TS'),
(503, 'C3', Sri, 'Chennai', 'TN'),
(504, 'C4', SD, 'Bangalore', 'KA');
```

-- DimRestaurant

```
INSERT INTO DimRestaurant (restaurant_key, restaurant_id, restaurant_name, cuisine, city)
VALUES
(20, 'R1', 'Meghana Foods', 'Andhra', 'Bangalore'),
(21, 'R2', 'Truffles', 'American', 'Bangalore'),
(22, 'R3', 'Paradise', 'Biryani', 'Hyderabad');
```

-- 4) Insert Fact Data --

-- FactFoodSales: 8 sample food sales rows

```
INSERT INTO FactFoodSales (sale_id, date_key, food_key, customer_key, restaurant_key, quantity,
amount) VALUES
(1, 20240101, 10, 501, 20, 2, 500.00), -- Srinivas bought 2 Chicken Biryani from Meghana
(2, 20240101, 12, 501, 20, 2, 120.00), -- Srinivas bought 2 Coke from Meghana
(3, 20240101, 11, 502, 21, 1, 180.00), -- Vas bought 1 Veg Burger from Truffles
(4, 20240102, 10, 504, 20, 1, 250.00), -- SD bought 1 Chicken Biryani from Meghana
(5, 20240201, 13, 503, 22, 2, 140.00), -- Sri bought 2 Gulab Jamun from Paradise
(6, 20240210, 10, 502, 22, 1, 260.00), -- Vas bought 1 Chicken Biryani from Paradise
(7, 20240305, 11, 501, 21, 2, 360.00), -- Srinivas bought 2 Veg Burger from Truffles
(8, 20240305, 12, 504, 21, 1, 60.00); -- SD bought 1 Coke from Truffles
```



-- 5) Analytical Queries (10) With Expected Answers

-- Q1) Overall total quantity and total sales amount

```
SELECT
    SUM(quantity) AS total_quantity,
    SUM(amount) AS total_sales_amount
FROM FactFoodSales;
```

-- Q2) Total sales amount by Month & Year

```
SELECT
    d.year,
    d.month_name,
    SUM(f.amount) AS monthly_sales
FROM FactFoodSales f
JOIN DimDate d ON f.date_key = d.date_key
GROUP BY d.year, d.month_name
ORDER BY d.year,
    FIELD(d.month_name,
        'Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec');
```

-- Q3) Top 3 food items by total revenue

```
SELECT
    fi.food_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimFoodItem fi ON f.food_key = fi.food_key
GROUP BY fi.food_name
ORDER BY total_revenue DESC
LIMIT 3;
```



-- Q4) Total revenue by food category

```
SELECT
    fi.category,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimFoodItem fi ON f.food_key = fi.food_key
GROUP BY fi.category
ORDER BY total_revenue DESC;
```

-- Q5) Total quantity and amount by Customer City

```
SELECT
    c.city,
    SUM(f.quantity) AS total_quantity,
    SUM(f.amount) AS total_amount
FROM FactFoodSales f
JOIN DimCustomer c ON f.customer_key = c.customer_key
GROUP BY c.city
ORDER BY total_amount DESC;
```

-- Q6) Restaurant-wise total sales amount

```
SELECT
    r.restaurant_name,
    SUM(f.amount) AS total_sales
FROM FactFoodSales f
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
GROUP BY r.restaurant_name
ORDER BY total_sales DESC;
```



-- Q7) Customer-wise total spend (Top customers)

```
SELECT
    c.customer_name,
    SUM(f.amount) AS total_spent
FROM FactFoodSales f
JOIN DimCustomer c ON f.customer_key = c.customer_key
GROUP BY c.customer_name
ORDER BY total_spent DESC;
```

-- Q8) Monthly revenue per Restaurant

```
SELECT
    r.restaurant_name,
    d.year,
    d.month_name,
    SUM(f.amount) AS monthly_restaurant_revenue
FROM FactFoodSales f
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
JOIN DimDate d ON f.date_key = d.date_key
GROUP BY r.restaurant_name, d.year, d.month_name
ORDER BY d.year,
    FIELD(d.month_name,
        'Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'),
    r.restaurant_name;
```

-- Q9) Cuisine-wise revenue

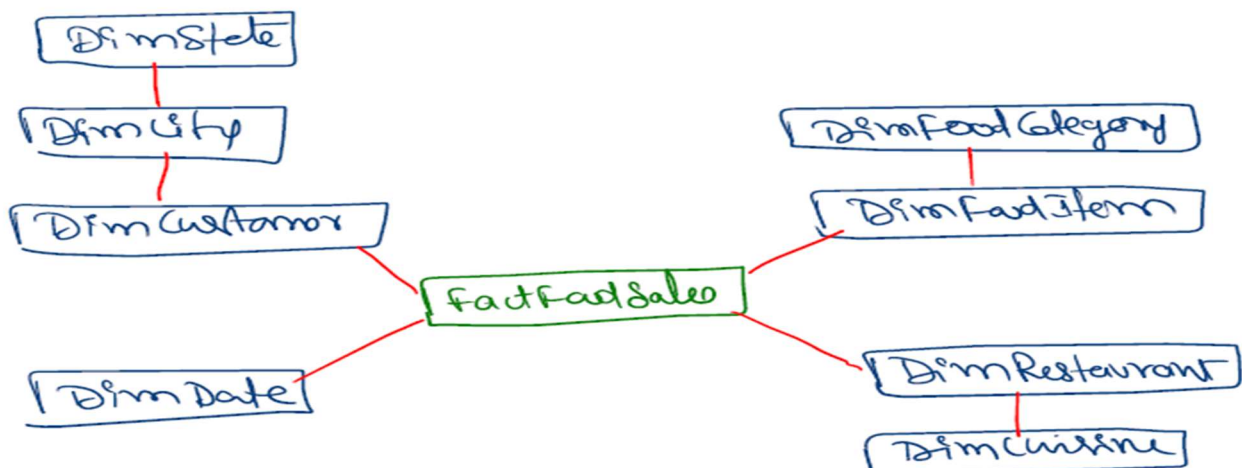
```
SELECT
    r.cuisine,
    SUM(f.amount) AS cuisine_revenue
FROM FactFoodSales f
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
GROUP BY r.cuisine
ORDER BY cuisine_revenue DESC;
```

-- Q10) Distinct customer count per Food Item

```
SELECT
    fi.food_name,
    COUNT(DISTINCT f.customer_key) AS distinct_customers
FROM FactFoodSales f
JOIN DimFoodItem fi ON f.food_key = fi.food_key
GROUP BY fi.food_name
ORDER BY distinct_customers DESC, fi.food_name;
```

4.7. Mini Case Study — Food Delivery Data Warehouse - SNOWFLAKE**SQL Design — Food Delivery SnowFlake Schema**

```
-- DROP TABLES (optional, for re-run)
-- DROP TABLE FactFoodSales;
-- DROP TABLE DimFoodItem;
-- DROP TABLE DimFoodCategory;
-- DROP TABLE DimCustomer;
-- DROP TABLE DimCity;
-- DROP TABLE DimState;
-- DROP TABLE DimRestaurant;
-- DROP TABLE DimCuisine;
-- DROP TABLE DimDate;
```





-- 1) Dimension Tables --

-- DimDate

```
CREATE TABLE DimDate (  
    date_key INT PRIMARY KEY,    -- e.g., 20240101  
    full_date DATE,  
    day_name VARCHAR(10),  
    month_name VARCHAR(10),  
    quarter VARCHAR(2),  
    year INT  
);
```

-- DimFoodCategory

```
CREATE TABLE DimFoodCategory (  
    category_key INT PRIMARY KEY,  
    category_name VARCHAR(50)  
);
```

-- DimFoodItem

```
CREATE TABLE DimFoodItem (  
    food_key INT PRIMARY KEY,  
    food_id VARCHAR(20),  
    food_name VARCHAR(100),  
    category_key INT,  
    FOREIGN KEY (category_key) REFERENCES DimFoodCategory(category_key)  
);
```

-- DimState

```
CREATE TABLE DimState (  
    state_key INT PRIMARY KEY,  
    state_name VARCHAR(50),  
    country VARCHAR(50)  
);
```



-- DimCity

```
CREATE TABLE DimCity (  
    city_key INT PRIMARY KEY,  
    city_name VARCHAR(50),  
    state_key INT,  
    FOREIGN KEY (state_key) REFERENCES DimState(state_key)  
);
```

-- DimCustomer

```
CREATE TABLE DimCustomer (  
    customer_key INT PRIMARY KEY,  
    customer_id VARCHAR(20),  
    customer_name VARCHAR(100),  
    city_key INT,  
    FOREIGN KEY (city_key) REFERENCES DimCity(city_key)  
);
```

-- DimCuisine

```
CREATE TABLE DimCuisine (  
    cuisine_key INT PRIMARY KEY,  
    cuisine_name VARCHAR(50)  
);
```

-- DimRestaurant

```
CREATE TABLE DimRestaurant (  
    restaurant_key INT PRIMARY KEY,  
    restaurant_id VARCHAR(20),  
    restaurant_name VARCHAR(100),  
    cuisine_key INT,  
    city_key INT,  
    FOREIGN KEY (cuisine_key) REFERENCES DimCuisine(cuisine_key),  
    FOREIGN KEY (city_key) REFERENCES DimCity(city_key)  
);
```



-- 2) Fact Table --

-- FactFoodSales (grain = 1 row per food item per order)

```
CREATE TABLE FactFoodSales (  
    sale_id      INT PRIMARY KEY,  
    date_key     INT,  
    food_key     INT,  
    customer_key INT,  
    restaurant_key INT,  
    quantity     INT,  
    amount       DECIMAL(12,2), -- line-level revenue  
  
    FOREIGN KEY (date_key)   REFERENCES DimDate(date_key),  
    FOREIGN KEY (food_key)   REFERENCES DimFoodItem(food_key),  
    FOREIGN KEY (customer_key) REFERENCES DimCustomer(customer_key),  
    FOREIGN KEY (restaurant_key) REFERENCES DimRestaurant(restaurant_key)  
);
```

-- 3) Insert Dim Data --

-- DimDate

```
INSERT INTO DimDate (date_key, full_date, day_name, month_name, quarter, year) VALUES  
(20240101, '2024-01-01', 'Monday', 'Jan', 'Q1', 2024),  
(20240102, '2024-01-02', 'Tuesday', 'Jan', 'Q1', 2024),  
(20240201, '2024-02-01', 'Thursday', 'Feb', 'Q1', 2024),  
(20240210, '2024-02-10', 'Saturday', 'Feb', 'Q1', 2024),  
(20240305, '2024-03-05', 'Tuesday', 'Mar', 'Q1', 2024);
```

-- DimFoodCategory

```
INSERT INTO DimFoodCategory (category_key, category_name) VALUES  
(101, 'Main Dish'),  
(102, 'Snack'),  
(103, 'Beverage'),  
(104, 'Dessert');
```



-- DimFoodItem

```
INSERT INTO DimFoodItem (food_key, food_id, food_name, category_key) VALUES
(10, 'F101', 'Chicken Biryani', 101),
(11, 'F102', 'Veg Burger', 102),
(12, 'F103', 'Coke', 103),
(13, 'F104', 'Gulab Jamun', 104);
```

-- DimState

```
INSERT INTO DimState (state_key, state_name, country) VALUES
(1, 'Karnataka', 'India'),
(2, 'Telangana', 'India'),
(3, 'Tamil Nadu', 'India');
```

-- DimCity

```
INSERT INTO DimCity (city_key, city_name, state_key) VALUES
(11, 'Bangalore', 1),
(12, 'Hyderabad', 2),
(13, 'Chennai', 3);
```

-- DimCustomer

```
INSERT INTO DimCustomer (customer_key, customer_id, customer_name, city_key) VALUES
(501, 'C1', 'Srinivas', 11), -- Bangalore, Karnataka
(502, 'C2', 'Vas', 12), -- Hyderabad, Telangana
(503, 'C3', 'Sri', 13), -- Chennai, Tamil Nadu
(504, 'C4', 'SD', 11); -- Bangalore, Karnataka
```

-- DimCuisine

```
INSERT INTO DimCuisine (cuisine_key, cuisine_name) VALUES
(201, 'Andhra'),
(202, 'American'),
(203, 'Biryani');
```



-- DimRestaurant

INSERT INTO DimRestaurant (restaurant_key, restaurant_id, restaurant_name, cuisine_key, city_key) VALUES

(20, 'R1', 'Meghana Foods', 201, 11), -- Andhra, Bangalore

(21, 'R2', 'Truffles', 202, 11), -- American, Bangalore

(22, 'R3', 'Paradise', 203, 12); -- Biryani, Hyderabad

-- 4) Insert Fact Data --

-- FactFoodSales: 8 sample food sales rows

INSERT INTO FactFoodSales (sale_id, date_key, food_key, customer_key, restaurant_key, quantity, amount) VALUES

(1, 20240101, 10, 501, 20, 2, 500.00), -- Srinivas bought 2 Chicken Biryani from Meghana

(2, 20240101, 12, 501, 20, 2, 120.00), -- Srinivas bought 2 Coke from Meghana

(3, 20240101, 11, 502, 21, 1, 180.00), -- Vas bought 1 Veg Burger from Truffles

(4, 20240102, 10, 504, 20, 1, 250.00), -- SD bought 1 Chicken Biryani from Meghana

(5, 20240201, 13, 503, 22, 2, 140.00), -- Sri bought 2 Gulab Jamun from Paradise

(6, 20240210, 10, 502, 22, 1, 260.00), -- Vas bought 1 Chicken Biryani from Paradise

(7, 20240305, 11, 501, 21, 2, 360.00), -- Srinivas bought 2 Veg Burger from Truffles

(8, 20240305, 12, 504, 21, 1, 60.00); -- SD bought 1 Coke from Truffles



-- 5) Analytical Queries (10) With Answers

-- 1) Category-wise revenue

```
SELECT
    fc.category_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimFoodItem fi ON f.food_key = fi.food_key
JOIN DimFoodCategory fc ON fi.category_key = fc.category_key
GROUP BY fc.category_name
ORDER BY total_revenue DESC;
```

-- 2) State-wise revenue

```
SELECT s.state_name, SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimCustomer cu ON f.customer_key = cu.customer_key
JOIN DimCity ci ON cu.city_key = ci.city_key
JOIN DimState s ON ci.state_key = s.state_key
GROUP BY s.state_name
ORDER BY total_revenue DESC;
```

-- 3) City + Category revenue

```
SELECT ci.city_name, fc.category_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimCustomer cu ON f.customer_key = cu.customer_key
JOIN DimCity ci ON cu.city_key = ci.city_key
JOIN DimFoodItem fi ON f.food_key = fi.food_key
JOIN DimFoodCategory fc ON fi.category_key = fc.category_key
GROUP BY ci.city_name, fc.category_name
ORDER BY ci.city_name, fc.category_name;
```



-- 4) Monthly revenue by State

```
SELECT
    d.year,
    d.month_name,
    s.state_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimDate d ON f.date_key = d.date_key
JOIN DimCustomer cu ON f.customer_key = cu.customer_key
JOIN DimCity ci ON cu.city_key = ci.city_key
JOIN DimState s ON ci.state_key = s.state_key
GROUP BY d.year, d.month_name, s.state_name
ORDER BY d.year,
    FIELD(d.month_name,
        'Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'),
    s.state_name;
```

-- 5) Cuisine-wise revenue

```
SELECT
    cu.cuisine_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
JOIN DimCuisine cu ON r.cuisine_key = cu.cuisine_key
GROUP BY cu.cuisine_name
ORDER BY total_revenue DESC;
```



-- 6) City-wise revenue

```
SELECT
    ci.city_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimCustomer cu ON f.customer_key = cu.customer_key
JOIN DimCity ci ON cu.city_key = ci.city_key
GROUP BY ci.city_name
ORDER BY total_revenue DESC;
```

-- 7) Restaurant-wise revenue

```
SELECT
    r.restaurant_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
GROUP BY r.restaurant_name
ORDER BY total_revenue DESC;
```

-- 8) Top 3 food items by revenue

```
SELECT
    fi.food_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimFoodItem fi ON f.food_key = fi.food_key
GROUP BY fi.food_name
ORDER BY total_revenue DESC
LIMIT 3;
```



-- 9) Top customers by total spend

```
SELECT
    cu.customer_name,
    SUM(f.amount) AS total_spent
FROM FactFoodSales f
JOIN DimCustomer cu ON f.customer_key = cu.customer_key
GROUP BY cu.customer_name
ORDER BY total_spent DESC;
```

-- 10) Monthly revenue by Cuisine

```
SELECT
    d.year,
    d.month_name,
    cu.cuisine_name,
    SUM(f.amount) AS total_revenue
FROM FactFoodSales f
JOIN DimDate d ON f.date_key = d.date_key
JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key
JOIN DimCuisine cu ON r.cuisine_key = cu.cuisine_key
GROUP BY d.year, d.month_name, cu.cuisine_name
ORDER BY d.year,
    FIELD(d.month_name,
        'Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'),
    cu.cuisine_name;
```



-- Top Restaurant(s) in each City by Total Revenue
-- Q11) Top 1 restaurant in each city (by revenue)

```
WITH city_restaurant_revenue AS (  
    SELECT  
        ci.city_name,  
        r.restaurant_name,  
        SUM(f.amount) AS total_revenue  
    FROM FactFoodSales f  
    JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key  
    JOIN DimCity ci ON r.city_key = ci.city_key  
    GROUP BY ci.city_name, r.restaurant_name  
)  
ranked AS (  
    SELECT  
        city_name,  
        restaurant_name,  
        total_revenue,  
        ROW_NUMBER() OVER (  
            PARTITION BY city_name  
            ORDER BY total_revenue DESC  
        ) AS rn  
    FROM city_restaurant_revenue  
)  
SELECT  
    city_name,  
    restaurant_name,  
    total_revenue  
FROM ranked  
WHERE rn = 1  
ORDER BY city_name;
```



-- Q12) Top 3 restaurants in each city (by revenue)

```
WITH city_restaurant_revenue AS (  
  SELECT  
    ci.city_name,  
    r.restaurant_name,  
    SUM(f.amount) AS total_revenue  
  FROM FactFoodSales f  
  JOIN DimRestaurant r ON f.restaurant_key = r.restaurant_key  
  JOIN DimCity ci ON r.city_key = ci.city_key  
  GROUP BY ci.city_name, r.restaurant_name  
)  
ranked AS (  
  SELECT  
    city_name,  
    restaurant_name,  
    total_revenue,  
    DENSE_RANK() OVER (  
      PARTITION BY city_name  
      ORDER BY total_revenue DESC  
    ) AS rnk  
  FROM city_restaurant_revenue  
)  
SELECT  
  city_name,  
  restaurant_name,  
  total_revenue,  
  rnk  
FROM ranked  
WHERE rnk <= 3  
ORDER BY city_name, rnk, restaurant_name;
```



5. Database Optimization

5.1. Database Partitioning

5.2. Data Sharding

5.3. Performance Anti-Patterns



5.1. Database Partitioning

What is Data Partitioning?

- Data partitioning is a technique used to divide a large dataset into smaller, manageable units called partitions.
- Each partition stores a subset of the data and can be managed independently.
- Partitioning helps databases scale efficiently as data volume grows.

Why Partition Data?

- **Enhances Scalability:**
 - ✓ Enables systems to handle very large datasets efficiently.
- **Improves Performance:**
 - ✓ Queries scan only relevant partitions instead of the entire table.
- **Balances Workload:**
 - ✓ Data and processing load are distributed evenly across partitions.
- **Improves Fault Tolerance:**
 - ✓ Failure in one partition does not affect the availability of other partitions.

Partitioning Types:

- Below are the commonly used partitioning approaches in large-scale systems:
 - ✓ Horizontal Partitioning
 - ✓ Vertical Partitioning
- Horizontal Partitioning
 - ✓ Divides a table row-wise, where each partition contains a subset of rows based on a condition (e.g., date, range, region).
- Vertical Partitioning
 - ✓ Divides a table column-wise, where each partition contains a subset of columns.



A) Horizontal Partitioning (Row-wise)

- Horizontal partitioning divides a table row-wise into multiple partitions.
- Each partition contains a subset of rows based on a partitioning key.
- Horizontal Partitioning is also called **Sharding**.

Horizontal Partitioning – Example

- Social Media User Database

Shard	User ID Range	Location
Shard 1	1 – 1M	United States
Shard 2	1M – 2M	Europe
Shard 3	2M – 3M	Asia

- **How It Works:**
 - ✓ User from the United States is stored in Shard 1
 - ✓ Queries related to U.S. users access only Shard 1, not the entire database
- **When to Use Horizontal Partitioning**
 - ✓ Applications with high-volume data
 - ✓ Systems with geographically distributed users
 - ✓ Scenarios requiring fast query performance by limiting data scans

B) Vertical Partitioning (Column-wise)

- Vertical partitioning divides a table column-wise.
- Each partition contains a subset of columns instead of rows.
- This approach is useful when different groups of columns are accessed with different frequencies.

Vertical Partitioning – Example

- E-Commerce Customer Table (Before Partitioning)

CID	Name	Location	Phone	Orders	Payments
101	Sri	NY, USA	12345	5	Credit Card
102	Vas	CA, USA	54321	3	PayPal
103	Hello	Blore, India	66666	2	UPI

- After Vertical Partitioning:
- **Partition 1 – Customer Profile (Frequently Accessed)**

CID	Name	Location	Phone
101	Sri	NY, USA	12345
102	Vas	CA, USA	54321
103	Hello	Blore, India	66666

- **Partition 2 – Orders & Payments (Less Frequently Accessed)**

CID	Orders	Payments
101	5	Credit Card
102	3	PayPal
103	2	UPI

- **How It Works:**
 - ✓ Customer profile queries access Partition 1
 - ✓ Order history queries access Partition 2

- **When to Use Vertical Partitioning**

- ✓ Tables with many columns
- ✓ When some columns are accessed frequently and others rarely
- ✓ To reduce I/O and improve cache efficiency

5.2. Data Sharding

- Data sharding is a distributed form of horizontal partitioning.
- It distributes data across multiple database instances or servers.
- Sharding is widely used in large-scale distributed systems to improve:
 - ✓ Scalability
 - ✓ Performance
 - ✓ Fault tolerance
- **Common Sharding Techniques:**
 - ✓ Range-based Sharding
 - ✓ Hash-based Sharding
 - ✓ Directory-based Sharding
 - ✓ Geographical Sharding
 - ✓ Dynamic Sharding
 - ✓ Hybrid Sharding
- Each approach balances data distribution, query efficiency, and operational complexity differently.



5.3. Performance Anti-Patterns

- Performance anti-patterns are common data modeling mistakes that may work initially but lead to serious performance and scalability issues as data grows.
- Understanding these helps prevent costly database redesigns later.
- Here are the List of Anti Patterns

1. Over-Normalization

- ✓ Creating too many small tables without considering query access patterns.
- ✓ **Problems:**
 - Excessive JOINS
 - Slower read performance
 - Increased query complexity
- ✓ **Example:**
 - Splitting user information into multiple tables even when always accessed together.

2. Excessive Joins in Core Queries

- ✓ Designing schemas that require many joins for frequent operations.
- ✓ **Problems:**
 - High query latency
 - Increased CPU and memory usage
 - Poor scalability
- ✓ **Example:**
 - orders → order_items → products → categories → suppliers

3. No Partitioning for Large Tables

- ✓ Keeping very large tables without partitioning.
- ✓ **Problems:**
 - Full table scans
 - Slower queries
 - Difficult maintenance
- ✓ **Example:**
 - An orders table growing by millions of records per month without partitioning.



4. Poor Primary Key Design

- ✓ Choosing keys that do not align with distribution patterns.
- ✓ **Problems:**
 - Hot partitions
 - Inefficient sharding
 - Slow lookups

5. Mixing OLTP and OLAP Workloads

- ✓ Using the same schema for transactions and analytics.
- ✓ **Problems:**
 - Transaction slowdown
 - Lock contention
 - Unpredictable performance

6. Multi-Valued Attributes in One Column

- ✓ Storing multiple values in a single column..
- ✓ **Problems:**
 - Difficult filtering
 - Index inefficiency
 - Data inconsistency
- ✓ **Example:**
 - items = 'burger,pizza,pasta'

7. Ignoring Data Growth Patterns

- ✓ Designing schemas without planning for long-term data growth.
- ✓ **Problems:**
 - Sudden performance degradation
 - Emergency schema changes

8. Overusing TEXT or BLOB Columns

- ✓ Storing large unstructured data in core transactional tables..
- ✓ **Problems:**
 - Increased I/O cost
 - Higher memory usage
 - Slower queries